

Situated Boundary Spanning: An Empirical Investigation of Requirements Engineering Practices in Product Family Development

RADHIKA JAIN, Baruch College

LAN CAO, Old Dominion University

KANNAN MOHAN, Baruch College

BALASUBRAMANIAM RAMESH, Georgia State University

Requirements Engineering (RE) faces considerable challenges that are often related to boundaries between various stakeholders involved in the software development process. These challenges may be addressed by boundary spanning practices. We examine how boundary spanning can be adapted to address RE challenges in Product Family Development (PFD), a context that involves complex RE. We study two different development approaches, namely, conventional and agile PFD, because these present considerably different challenges. Our findings from a multisite case study present boundary spanning as a solution to improve the quality of RE processes and highlight interesting differences in how boundary spanner roles and boundary objects are adapted in conventional and agile PFD.

Categories and Subject Descriptors: K.6.3 [Software Management]: Software Process

General Terms: Design, Management, Human Factors

Additional Key Words and Phrases: Boundary spanning, requirements engineering, product family development, agile development

ACM Reference Format:

Radhika Jain, Lan Cao, Kannan Mohan, and Balasubramaniam Ramesh. 2014. Situated boundary spanning: An empirical investigation of requirements engineering practices in product family development. *ACM Trans. Manag. Inform. Syst.* 5, 3, Article 16 (December 2014), 29 pages.
DOI: <http://dx.doi.org/10.1145/2629395>

1. INTRODUCTION

Despite significant developments in practices, tools, and techniques, Requirements Engineering (RE) continues to face significant challenges in five facets of RE: elicitation, analysis, modeling, verification/validation, and management [Clements and Northrop 2002]. Many of the challenges faced in RE result from the need to involve multiple stakeholder groups separated by various “fault lines”: hypothetical lines such as domain expertise, affiliation, knowledge domain, occupational roles, professional experience, and other demographics [Gasson 2007; Lau and Murnighan 2005; Slaughter et al. 2007]. The presence of such fault lines may result in a lack of shared goals, shared culture, shared process, shared responsibility, and trust that negatively affects the RE process [Bhat et al. 2006]. Fault lines have been examined in the literature on *boundary spanning*, which is the process through which organizational stakeholders in different fields of practice relate their practices to other fields in order to develop

Authors' addresses: R. Jain and K. Mohan, Department of Statistics & Computer Information Systems, Baruch College; L. Cao, Department of Information Technology & Decision Sciences, Old Dominion University; B. Ramesh, Department of Computer Information Systems, Georgia State University.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies show this notice on the first page or initial screen of a display along with the full citation. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers, to redistribute to lists, or to use any component of this work in other works requires prior specific permission and/or a fee. Permissions may be requested from Publications Dept., ACM, Inc., 2 Penn Plaza, Suite 701, New York, NY 10121-0701 USA, fax +1 (212) 869-0481, or permissions@acm.org.

© 2014 ACM 2158-656X/2014/12-ART16 \$15.00

DOI: <http://dx.doi.org/10.1145/2629395>

knowledge-based competence [Carlile 2002; Levina and Vaast 2005]. Roles performed by boundary spanners and boundary objects can help bridge fault lines across different “fields of practice” [Levina and Vaast 2005]. Despite the close connection between boundary spanning and RE, prior literature has not examined how boundary spanning can mitigate RE challenges and improve the RE process.

One of the key challenges in RE pertains to the anticipation and representation of emergent behaviors of the system [Jarke et al. 2011]. To explore this further, we consider Product Family Development (PFD) as a context wherein RE is significantly more complex and the emergent nature of requirements is more pronounced than in a non-PFD software development environment. Rather than developing several products in isolation from one another, the family-based approach emphasizes the exploitation of commonality across these products [Clements and Northrop 2002].

Software development organizations that choose PFD could resort to two development approaches: conventional PFD and agile PFD [Ghanam et al. 2012; Mohan et al. 2010]. We consider these two approaches as excellent candidates to explore how boundary spanning practices address RE challenges. In conventional PFD, RE focuses on understanding up front the commonality and variability in requirements across customers in a market segment. The goal is to develop a stable product platform that satisfies requirements that are common across different customer segments. Although this approach helps to deliver a variety of products with a shorter turnaround time, it requires considerable up-front investment of time and resources. It is characterized by fault lines pertaining to the knowledge boundary between client and development teams and fault lines pertaining to the functional boundary between domain and application engineering teams. This poses a stark contrast with agile PFD in which RE exhibits a blend of emphasis on building a product platform to support speedy development of product variants as well as responsiveness to changing requirements [Cockburn and Highsmith 2001]. Rather than requiring significant up-front investment to accommodate anticipated changes, agile PFD focuses on critical requirements that are important in the short term. The need to bridge the fault lines pertaining to the knowledge boundary between the customer and the team are more pronounced because requirements emerge continually from the interactions between the two groups.

These critical differences in RE between agile and conventional PFD may lead to differences in the boundary spanning practices used in each approach. Our goal is to understand how boundary spanning addresses the RE challenges faced in the PFD context and, as a result, improves the quality of the RE process. This leads us to the research question: “How do boundary spanning practices help improve the quality of RE process in agile and conventional PFD?” To answer this question, we conducted a multisite case study of two agile and two conventional PFD projects.

The rest of this article is organized as follows: We first introduce product family development and then present RE challenges identified in prior research. We then draw from the literature on boundary spanning and highlight its connection to RE. The research methodology section presents details on case study sites, data collection, and data analysis procedures. We then present our findings, which explain how adapted boundary spanning practices enable RE in conventional and agile PFD processes. We discuss these adapted boundary spanning practices by elaborating on their characteristics. Finally, in the discussion section, we highlight our theoretical and practical contributions.

2. THEORETICAL BACKGROUND

2.1. Product Family Development and Requirement Engineering

A software product family (also known as software product line) is a set of software-intensive systems that share a common, managed set of features [Clements and

Northrop 2002]. It is a set of products that have common aspects and predicted variations that satisfy the specific needs of a particular market segment or mission [Weiss and Lai 1999]. PFD includes two steps: domain engineering and application engineering. Domain engineering encompasses domain analysis—a process for capturing and representing information about applications in a domain, specifically common characteristics and reasons for variability [Bosch et al. 2001]. Application engineering involves the development of individual products by selecting and configuring the core assets developed during domain engineering and adding product-specific extensions.

PFD presents an important context that characterizes the need to anticipate and represent evolving elements of the business domain. Anticipating and incorporating such business needs into the product platform and the ability to derive product variants from the platform requires a mature RE process in PFD. The need for understanding the common and variable requirements and the relationship between them increases the complexity of the RE process in PFD. This is particularly interesting when we consider two approaches that organizations take toward PFD—conventional and agile—because they differ in how they address the RE challenges discussed herein.

2.2. RE Challenges

Challenges faced in traditional non-PFD environments for RE have been attributed to lack of shared goals, shared culture, shared process, shared responsibility, and trust [Bhat et al. 2006]. These challenges have certain peculiarities in PFD [Clements and Northrop 2002; Moon et al. 2005]. RE challenges in PFD owe their complexity to the management of commonality and variability [Alves et al. 2010]. For example, characteristics of requirements (mandatory vs. optional in some product variants), increased dependencies across requirements, and variations at different levels of details [Eriksson et al. 2009] make RE more complex in PFD. We discuss how various challenges are amplified in PFD in five facets of RE: elicitation, analysis, specification, verification/validation, and management.

2.2.1. Requirement Elicitation. In requirement elicitation, engaging the customer base is a major challenge. Inadequate customer representation [Lawrence et al. 2001] and inadequate user–developer interaction [Curtis et al. 1988; Keil et al. 1998; Kraut and Streeter 1995] lead to difficulties in identifying the right requirements. When user communities are fragmented, it is difficult to obtain shared understanding and, more importantly, concurrence among different customer segments. These problems are amplified in PFD because developers need to anticipate and accommodate changes that might affect different customer segments. The architecture of the common platform needs to elegantly accommodate the anticipated changes. Given the diversity in target customer segments, the number of stakeholders involved is large [Alves et al. 2010]. Challenges to RE in PFD range from making the common and variable requirements transparent to the customers to ensuring that customers understand the implications of variable requirements. RE in an agile environment is even more challenging due to the difficulty involved in obtaining the needed level of customer access and participation [Nerur et al. 2005].

2.2.2. Requirement Analysis. In requirement analysis, understanding requirements properly is a major challenge because there is always the risk of overlooking crucial requirements [Lawrence et al. 2001]. A risk that is peculiar to PFD is a failure to distinguish between common and variable requirements [Clements and Northrop 2002]. Insufficient or excessive generality in requirements may lead to problems in both domain and application engineering. In agile development, customer involvement improves the understandability of the requirements; however, frequent changes of project

scope and feature priorities may cause problems, such as instability of the platform [Ramesh et al. 2010].

2.2.3. Requirement Specification. In requirement specification, an important challenge is to present requirements without allowing design to creep into requirements specifications [Lawrence et al. 2001]. In PFD, this becomes even more challenging because of the need to document common and variable requirements while also identifying mandatory and optional features in both the platform and specific product variants [Clements and Northrop 2002]. The ability to specify and communicate the details about the common and variable requirements with all the stakeholders is critical. In agile environments, the lack of detailed requirements documentation may cause problems if communication breakdown occurs due to personnel turnover and inaccessibility of the customer [Rubin and Rubin 2012].

2.2.4. Requirement Validation/Verification. In requirement validation/verification, uncertainty exists in evaluating requirements' correctness and completeness [Davis 1982] and inspecting requirements [Lawrence et al. 2001]. In PFD, commonality and variability increase the complexity of verification because of the need to verify both common and variable requirements. This also entails the analysis of any inconsistencies between common and variable requirements. Ensuring consistency and completeness of requirements is very difficult in PFD [Padmanabhan and Lutz 2005] due to the increased dependencies across variable requirements and between common and variable requirements. In agile development, requirements validation is done through frequent feedback from the customer [Kovitz 2003]. Some forms of requirements verification are challenging in agile development due to the lack of formal modeling [Paetsch and Maurer 2003].

2.2.5. Requirement Management. Finally, challenges in handling requirement volatility are well recognized [Davis 1982; Demarco and Lister 2003; Keil et al. 1998]. In PFD, requirements management increases in complexity due to the need to manage changes in platform- and product-specific requirements, both of which require the participation of stakeholders from different backgrounds and thus create the potential for fault lines. In agile development, although RE is less documentation-oriented, requirements volatility is well managed by agile practices such as short development cycles and constant communication with the customers [Conboy and Morgan 2011].

2.3. RE, Boundaries, Boundary Spanner Roles, and Boundary Objects

The quality of the RE process largely depends on how well the challenges in these five RE facets are addressed. Most of the challenges in RE arise from the need to involve multiple stakeholder groups that are separated by various boundaries. Boundaries need not be just based on functional differences, but also on other factors such as geographical distance, time, language, culture, status, intellectual, social, and economic differences [Espinosa et al. 2003]. The presence of these boundaries lets each subunit or context develop its own culture—local norms, values, and languages [Polzer et al. 2006]. This may result in reduced collaboration among stakeholders [Levina and Vaast 2008]. Social boundaries may lead to challenges in developing shared identities and practices [Levina and Vaast 2005]. The need to examine how these boundaries that separate the various stakeholders in PFD projects can be traversed aligns well with the findings from the literature on boundary spanning. Yet little prior research has been done to integrate boundary spanning with RE. Boundary spanning research has focused on studying social interactions of diverse stakeholder groups in the presence of such boundaries, thus providing an appropriate theoretical lens to study how RE challenges may be mitigated. Specifically, the literature suggests boundary objects and

boundary spanner roles as two important facets of boundary spanning that can help bridge fault lines pertaining to such boundaries between stakeholders. We draw on this literature on boundaries, boundary spanner roles, and boundary objects in using boundary spanning as the theoretical lens for our study.

2.3.1. Boundary Spanner Roles. Roles of boundary spanners range from ambassadors (who engage in political activities such as lobbying for support and resources and buffering a group from outside pressure), task coordinators (who facilitate various activities), and scouts (who engage in environmental scanning activities) [Ancona and Caldwell 1992; Yan and Louis 1999]. Key boundary spanning activities include knowledge acquisition, negotiation [Kirsch and Haney 2006], consensus building, and conflict resolution [Carlile 2002, 2004]. In the context of knowledge transfer, Carlile [2002, 2004] identifies three primary boundary spanning acts: transfer, translation, and transformation. Whereas knowledge transfer relies on a common lexicon shared by those separated by boundaries, knowledge translation is intended to address “interpretive differences” across those separated by boundaries [Carlile 2004]. Knowledge transformation emphasizes the differing interests of actors separated by boundaries. Boundary spanning may also involve playing the role of a knowledge broker [Pawłowski and Robey 2004]. Knowledge brokers gain access to and work within many different knowledge domains and connect otherwise disconnected actors by enabling flow of resources or information, and they “shape collegial and managerial perceptions” [Fleming and Waguespack 2007]. Their role is to not only enable movement of knowledge (i.e., translate and transfer) but also to identify, localize, redistribute, mediate, and transform knowledge to meet the differing needs of users from different domains by producing brokered knowledge that is robust, accountable, usable, and locally relevant [Meyer 2010].

The use of boundary spanning practices situated in the unique contexts of RE activities has not received much attention [Henfridsson and Lindgren 2010]. Boundary spanning in RE becomes particularly difficult when important and powerful stakeholders, such as clients or management teams, participate only occasionally, causing potential disruptions to the process of forming consistent understanding of requirements [Davidson 2002]. In the current research, we seek to understand how boundary spanning improves the quality of the RE process in PFD.

2.3.2. Boundary Objects. Boundary objects can be viewed as means of “representing, learning about, and transforming knowledge to resolve the consequences that exist at a given boundary” [Carlile 2002]. Boundary objects mediate interactions between different communities of practice by providing practical and political means to bridge the boundaries and develop a common basis for dialogues about solutions to problems [Dodgson et al. 2007].

Carlile [2002, 2004] classifies boundary objects into three categories—syntactic, semantic, and pragmatic—according to the type of boundary each is associated with. Boundary objects examined in IS literature range from shared IT systems [Pawłowski and Robey 2004], simulation technologies [Dodgson et al. 2007], documents in a common repository in virtual workspaces [Malhotra and Majchrzak 2012], and 3-D architectural models [Gal et al. 2008] to design drawings and prototypes to various types of information systems [Levina and Vaast 2005]. Although effective boundary objects are tangible, concrete, accessible, and up-to-date, their usefulness and performance also depend on how spanners mobilize their social, economic, and symbolic capital [Levina and Vaast 2005]. How people “mobilize” boundary objects (i.e., how actors contribute their resources, either material or nonmaterial such as knowledge and time), integrate, and coordinate them [McEvily et al. 2003] determines whether boundary objects promote or inhibit shared understanding [Bechky 2003]. Boundary objects such as project plans and architectures in design ecologies can help minimize gaps in

the design knowledge held in functional ecologies as well as reduce disagreements in political ecologies [Bergman et al. 2007]. To achieve this, Bergman et al. [2007] suggest that boundary objects used in design ecologies should (i) promote common representation, (ii) transform design knowledge, (iii) mobilize for design action, and (iv) legitimize design knowledge across functional and political ecologies. Through an analysis of the use of boundary objects, Gasson [2005] suggests that new uses of boundary objects signal a change in the focus of communication among stakeholders. Gasson [2005] also identifies the need to adapt boundary objects depending on the specific concerns of designers. The distinction between “closed” and “open” boundary objects is important in supporting design processes [Nandhakumar et al. 2013] because they indicate whether boundary objects are amenable to evolution based on the questions they raise rather than on the answers they provide.

To summarize, to address RE challenges in PFD, spanning boundaries that separate diverse stakeholder groups are essential. Extant research has not examined how boundary objects and the roles of boundary spanners are adapted to improve the quality of the RE process, specifically within the context of PFD.

3. RESEARCH METHODOLOGY

We conducted a multiple-case study [Yin 2003] of one PFD project each in four organizations. Through multisite case studies, we were able to compare practices across conventional and agile PFD environment. Although the literature on RE and boundary spanning is extensive, there is a paucity of research that focuses on understanding how boundary spanning addresses RE challenges in PFD. A case study approach is most suitable for research that raises “why and how” questions and is appropriate to inductively develop a framework [Eisenhardt 1989] that explains how boundary spanning practices are used in the RE process in PFD.

3.1. Research Sites

Two of the four organizations, Alpha and Beta, follow conventional PFD, whereas two others, Gamma and Delta, follow agile PFD. The primary criterion was the research sites’ potential to support the investigation of how boundary spanning practices were used to address RE challenges in PFD. Our focus was not on selecting organizations that are representative for us to generalize our results, but on the opportunity to generate a nuanced understanding of the role of boundary spanning in addressing RE challenges in our study context. Our selection of each case was driven by whether it permitted us to replicate or extend the emergent theory [Eisenhardt and Graebner 2007]. The dimensions that we used to select the cases are: (i) the project involved product family development, (ii) the project followed the tenets of the specific approach (conventional or agile), and (iii) the project demonstrated RE challenges. We selected two sites that were engaged in three activities of core asset development, product development, and technical and organizational management [Clements and Northrop 2002] that are characteristic of conventional PFD. The two agile PFD projects exhibited characteristics consistent with Conboy’s [2009] definition of agility in ISD: They rapidly embraced the changes in requirements presented by the customer and used customer feedback to learn about customer requirements. Furthermore, the customers perceived the projects as providing value in terms of meeting their current needs with expected levels of quality. The two groups of organizations provided us a relevant theoretical organizational, environmental, and IS development context for studying the phenomenon of interest.

3.1.1. Alpha. Alpha is a provider of supply chain management solutions. It develops families of Warehouse Management Systems (WMS) for various markets in

food/grocery, retail, transportation, and life sciences. It developed a product platform that encompassed most of the functionality required by its initial customer base. Alpha has gradually expanded this platform to satisfy commonly demanded requirements. It has a domain engineering team whose responsibility is the development and maintenance of the product platform and application engineering teams that work closely with specific clients to derive product variants from the platform to cater to the unique needs of their respective clients. Alpha follows a conventional approach to PFD, with considerable emphasis on a stable and adaptable platform and mature practices in product derivation. Alpha's products have been very successful in the market and have continued to perform well.

3.1.2. Beta. Beta is in the forest, paper, and packaging industry with hundreds of manufacturing and distribution facilities throughout North America. It developed diverse product lines and consumer markets within this industry. It was organized into highly diversified business divisions, which were virtually autonomously managed. Recognizing inefficiencies due to the lack of a common platform, information sharing, and coordination across its business divisions, Beta established a Corporate Program Office (CPO) team to develop the specifications for a common platform for transportation planning processes in all divisions. Beta followed a conventional PFD approach to transition to this common platform. This platform covered various transportation planning functionalities such as route optimization, freight consolidation and payment, load optimization, load tendering, carrier management, and shipment tracking. All the involved stakeholders deemed this initiative to develop the common platform successful.

3.1.3. Gamma. Gamma provides IT development and consulting services worldwide. The focal project of this study involved the development of a subsystem in a large suite of a network security software product line. The customers include the world's largest banks, governments, and top IT organizations. Gamma had developed a platform with common functionalities. The focal project team adopted eXtreme Programming (XP) in developing a product variant. The customer of the project, which was another development group within Gamma, demanded a flexible approach in which project scope and requirements were not frozen up front but were emerging throughout the process. A product manager served as the customer representative and was on site to provide continuous feedback and inputs to the team. The application was successfully delivered on time, and both the project manager and the customer were satisfied with the quality of the product.

3.1.4. Delta. Delta provides a variety of consumer products to both wholesale and retail clients. The focal project involved deriving a variant from a product platform for tracking shipments of various goods. A platform development team, which was part of another development unit, created the core assets that provided the common functionalities. The focal project team adapted the core platform to the specific needs of one client group. A product management team interacted with different client groups and acted as a product owner for each of the variations derived from the product platform. The team adopted all XP and Scrum practices. The product manager representing the client market segment suggested that the project met all its major goals in terms of delivering the necessary functionality within cost and budget constraints.

3.2. Data Collection

We collected qualitative data from the four sites through interviews, emails, observations, and other documentation. We used various data collection strategies [Lee 1989]

Table I. Summary of Data Collection at Four Sites

Site	Primary data sources	# of contacts	Roles	Archival documents examined
Alpha	Interviews	12	Informants from alpha played multiple roles in their project. These roles included business analysts, architects, and programmers.	Reports from a change management system, snapshots of how code bases are managed in a configuration management system
Beta	Workshops and meetings	14	CIO, initiative sponsor, senior executives that included the vice presidents and IT directors, CPO team members	Process model diagrams, spreadsheets of process analysis documents, summary notes of CPO team, PowerPoint presentations
Gamma	Interviews	5	Project manager, product manager (customer representative), developer, project lead, tester	User stories, iteration planning documents
Delta	Interviews	14	Product owner (customer representative) project manager/scrum master, developers, project team lead, senior manager	Burn down charts, sprint backlogs, product backlog
Total		45		

to ensure continuous engagement with the research sites to collect rich qualitative data.

The primary source of data was semistructured interviews with individual informants at Alpha, Gamma, and Delta. Senior managers in the focal organizations helped us identify informants who were involved in RE. They, in turn, helped us reach additional informants (see Table I). At Beta, data were collected during eight workshops in which 10–15 senior executives (such as vice presidents and IT directors of transportation planning processes) participated and also during six pre-workshop meetings that only involved members of CPO team.

The interviews at Alpha, Gamma, and Delta and the workshop sessions at Beta were audio taped and transcribed. Interviews typically lasted 90 minutes, although a few ran as long as 3 hours. Copious notes also were taken during the interviews and workshops, and these were integrated with the transcripts of the recordings. During site visits, we also recorded informal observations from activities such as group meetings, product demonstrations, and status reviews. Furthermore, we also examined different types of documentation to develop an understanding of boundary spanning in the focal projects and to triangulate our findings.

3.3. Data Analysis

Transcripts of interviews and workshop meetings were used as the primary data source in this study. We analyzed each case separately. Text fragments from the interview transcripts were tagged with one or more codes, along with justifications for the selection of codes. We coded the interview data based on the initial set of seed concepts drawn from boundary spanning literature: boundary, boundary objects, and boundary spanner roles. The data were coded based on how they fit the definitions of the seed concepts. For example, requirements specification documents were boundary objects used to bridge discontinuities across different stakeholder groups. Boundary

spanner roles were coded based on the view that individuals leveraged their capital from their field of practice to link to a new or different field of practice. For example, some senior managers served as ambassadors for their projects. Concepts that emerged from our data analysis are connected to these seed concepts. For example, the practice in our framework “Creating controlled boundaries” is a boundary spanning practice that is also connected to the characteristics of a boundary that is affected (i.e., it is an instance of a boundary spanning practice that highlights how boundary spanning is adapted to shape a particular aspect of a boundary).

Two researchers separately examined each case for evidence. For each case, the authors recorded information about boundary spanning practices. Each author analyzed the data independently, and then the all the authors combined their analyses by resolving discrepancies between each other’s entries through discussion, referencing back to the interview reports, and consulting with another author. Once the individual case analyses were complete, we used a cross-case analysis to compare the cases across conventional and agile PFD [Eisenhardt 1989; Miles and Huberman 1994] to develop the conceptual insights by examining similarities and differences. The theoretical concepts that emerged from the data are summarized in the conceptual framework in Figure 1 and described in the next section. During the data analysis process, plausibility and consistency of the concepts, subcategories, and links were established. This helped to ensure that our conceptual framework satisfied established criteria for the credibility and validity in qualitative research [Miles and Huberman 1994; Pawlowski and Robey 2004]. Although we do not attempt to statistically evaluate the strength of concepts included in the framework, theoretical relevance of concepts is indicated by their repeated presence or notable absence when comparing incidents [Strauss and Corbin 1990].

To establish credibility, which is similar to internal validity in quantitative studies [Guba 1981], we performed triangulation by using multiple sources and/or multiple investigators [Mason 2002]. We iteratively cross-examined evidence from multiple informants to ensure that categories were well developed and represented multiple viewpoints. The presence of multiple investigators allowed us to systematically recognize, discuss, and debate different interpretations and improve our understanding of the RE challenges faced by the informants and various boundary spanning practices that were developed to address the RE challenges. To enhance the internal validity and generalizability [Orlikowski 1993; Strauss and Corbin 1990], we relate our findings to existing literature on boundary spanning, technical debt, and PFD.

4. FINDINGS

4.1. Our Framework

Our research framework (shown in Figure 1), developed based on the analysis of data and the literature on RE, presents various boundary spanning practices that were observed in conventional and agile PFD projects. These practices address several RE challenges and thus contribute to improving the quality of the RE process.

Table II summarizes our findings on how various boundary spanning practices across conventional and agile PFD help mitigate RE challenges discussed in the literature section. We also summarize the elements of boundary spanning (i.e. boundaries, spanner roles, and boundary objects) across the four cases in Table III. It identifies the different boundaries pertaining to various types of fault lines observed in our cases. For example, at Alpha, a knowledge boundary pertains to the fault line between the client and the domain/application engineering teams’ areas of expertise. The client possesses domain expertise, and the development team possesses system and development process expertise.

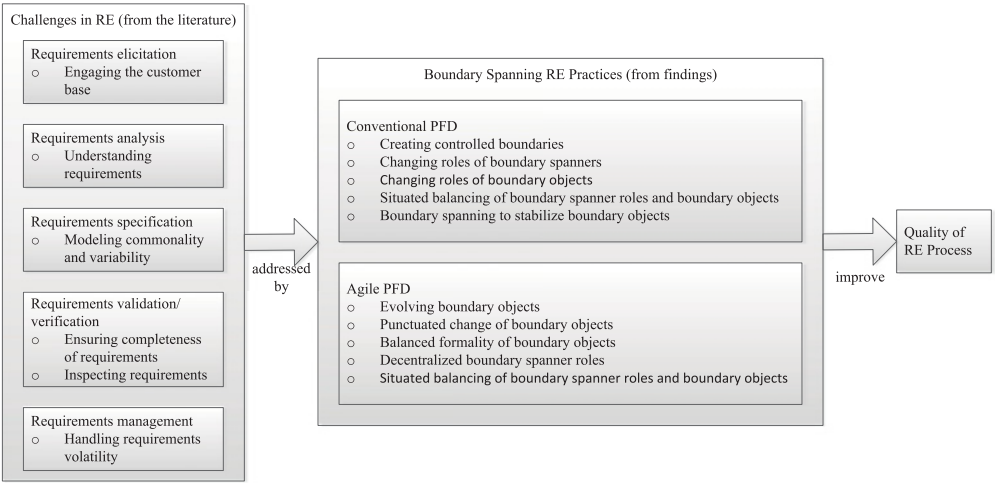


Fig. 1. Research framework based on analysis of data from multiple sites.

Table II. Mapping How RE Challenges Are Addressed by Various Boundary Spanning Practices

RE Challenges	Addressed by boundary spanning practices in	
	Conventional PFD	Agile PFD
Requirements Elicitation	Changing roles of boundary spanners	Decentralized boundary spanners
Requirements Analysis	Controlled boundaries	Evolving boundary objects Punctuated change of boundary objects
Requirements Specification	Situated balancing of boundary spanner roles and boundary objects Boundary spanning to stabilize boundary objects	Balanced formality of boundary objects
Requirements Validation/Verification	Changing roles of boundary spanners	Evolving boundary objects
Requirements Management	Changing roles of boundary objects	Punctuated change of boundary objects Balanced formality of boundary objects

The elements of the framework (Figure 1) are described in the following sections by first describing each practice with examples from the cases. We end the discussion of each approach to PFD by linking the practices to the RE challenges.

4.2. Boundary Spanning in Conventional PFD

4.2.1. *Creating Controlled Boundaries.* *Creating controlled boundaries* focuses on how various stakeholders—division stakeholders, CPO team, customers, application engineers, and domain engineers—traversed the boundaries among them so as to balance each of their needs. A controlled boundary refers to the boundary that is monitored, and the artifacts that cross it are carefully examined and filtered by the stakeholders involved. Although customer requirements had to be incorporated either as a variation or as part of the platform, domain and application engineers worked in tandem to ensure that they had a stable platform that delivers value across all market segments. In other words, the boundary between application and domain engineering teams was

Table III. Summary of Boundary Spanning Elements

	Alpha	Beta	Gamma	Delta
Teams involved	• Domain and application engineering teams • Client	• CPO team • Process stakeholders • Top management team	• Application team • Customer representatives • Product family maintenance team	• Application development team, • Customer representatives • Senior executives
Team activities	• Understand, analyze, and model common and variable requirements (bottom-up approach) • Develop and maintain product platform • Develop customizations from the platform	• Understand transportation planning (TP) processes and supporting systems for each division • Identify common and variable TP activities across all divisions • Identify mechanism to manage variability	• Develop a suite of security applications that constitute product variants derived from the platform • Update platform requirements	• Develop shipment tracking variants • Update platform requirements
Boundaries	• Functional boundary pertaining to fault-line between domain and application engineering • Knowledge boundary pertaining to fault-line between client and domain/application engineering	• Knowledge boundary pertaining to fault-line between process stakeholders from different divisions • Functional boundary pertaining to fault-line between process stakeholders and top management team	• Knowledge boundary pertaining to fault-line between customer and the application team • Functional boundary pertaining to fault-line between the team members	• Knowledge boundary pertaining to fault-line between customer and the application team • Functional boundary pertaining to fault-line among the team members • Functional boundary pertaining to fault-line between the development team and senior management
Boundary objects	• Traceability documentation	• Process models • Vocabularies	• User stories • Product/process repositories • XP practices • Feature models of platform	• Product backlog • Sprint backlog • Product prototypes
Boundary spanner roles	• Filters/gatekeepers • Ambassadors	• Ambassadors • Scout • Knowledge integrators	• All members serve as boundary spanners • Gatekeeper of project manager • Knowledge broker • Task coordinator	• All members serve as boundary spanners • Ambassador • Gatekeeper • Knowledge broker • Task coordinator

controlled to ensure that requirements flowing back and forth did not adversely affect the platform. Similarly, the CPO team worked closely with division stakeholders to ensure that stakeholders were able to identify and justify division-specific features while striving to identify as much commonality across the divisions as possible to enable a stable platform.

Recognizing that a comprehensive domain analysis involves significant investments in terms of both time and other resources that may delay the time-to-market and increase the cost of an initial set of products, Alpha decided to carry out this activity incrementally. It initially restricted domain analysis to a single vertical industry—the apparel industry—and later expanded to include other industries. The tightly

controlled boundary between domain and application engineering ensured that the platform requirements were stabilized.

During the domain analysis process, Alpha also made a conscious decision not to attempt to address the requirements of all of its customer segments. They decided to focus on the core functionalities desired by most of their customers to achieve a quick consensus on the critical requirements that would be initially implemented in the platform. As a result, Alpha was able to complete domain engineering activities and implement the platform at a much faster pace and with limited resources. *“The engine of the whole product will be common across all the verticals. . . . It will be much more efficient and cost effective if we do it this way and if most customers feel so then that practice will be implemented in the product.”* [Manager, Alpha]

At Beta, the guiding principle of “Nothing off the table” drove each division to rethink and rationalize any unique aspects characterizing their planning processes. Process stakeholders were instructed to identify business rationale for division-specific exceptions to create variations to the base platform. *“We also want all the process owners to do a cost-benefit analysis when they ask for any new functionality to be added. It’s just fair to do it that way as we have to listen to everyone.”* [CPO facilitator, Beta] This highlighted the desire to achieve a balance between features incorporated in the platform and the product variants. As process stakeholders struggled to achieve such balance, the CPO team developed an elaborate process-modeling template to facilitate this task. This template captured contextual elements underlying the planning processes in terms of business rules, decisions, alternatives, assumptions, inputs, outputs, exceptions, systems used, and process gaps. Using this template, process stakeholders added contextual details to uncover and better articulate various hidden assumptions. This step speeded the process of validating the rationale behind the unique requirements of each division.

4.2.2. Changing Roles of Boundary Spanners. The practice of *changing roles of boundary spanners* indicates that certain members of the domain engineering teams had to play different roles in shaping a stable platform. They were insulated from application engineering teams to avoid being overwhelmed by too many requirements and/or changes to the platform. However, they had to explain the coverage of the platform to the customers and other stakeholders to encourage them to consider adopting the platform with fewer variable requirements.

Although the benefits of incorporating functionality (and achieving planned reuse) through the product platform were well recognized, Alpha was also careful not to inundate the platform with features that were not applicable to a significant majority of their clients. *“When it goes to the base, the product development team will put a lot of thought into it and make sure that it can serve more than one client. They will make sure that this can be expanded, and it will have much more added functionality in everything and not specific to one client.”* [Manager, Alpha] Although the introduction of functionalities into the platform increases reuse, it also results in a significant increase in platform maintenance costs. When application developers identify the need for potential changes in the platform based on the prevalence of common features across different variants, they follow a formal process to alert the domain engineering team. This team then carefully examines the extent to which the proposed changes will affect various customer segments. The diversity in the customer base implies that any changes in the platform could have profound impacts on a product variant that is significantly different from other variants. This requires that developers who have significant knowledge about the platform and client-specific variants perform dual roles. On the one hand, they encourage clients to use as much of the platform’s functionality as possible, whereas, on the other hand, they were also protecting the platform from unnecessary changes.

At Beta, the role of the CPO team evolved significantly as it facilitated the tasks of requirements elicitation, analysis, and modeling activities. The CPO team took on various roles, such as being ambassadors, scouts, and coordinators who helped the process stakeholders at different stages of deliberations to identify common and variable requirements. In these roles, the CPO team addressed many of the shortcomings that had plagued prior attempts at requirements determination in platform development. In the initial stages of the initiative, the CPO team acted as champions for the initiative, pointing out to stakeholders how different divisions had continued to operate in an isolated fashion that led to lack of synergies: *“There is no opportunity across [Beta] to really leverage our collaborative bases, and it is also costing us a lot of money to do what we do now. There is a huge payback for [Beta].”* [CPO member-1, Beta] The CPO team in its ambassadorial role negotiated with individual divisions to bring the empowered process stakeholders to extensive deliberations to identify common platform features. It effectively bridged the communication gap between the top management team and process stakeholders during many of these negotiations and secured the participation of all divisions. As the initiative progressed with buy-in from all the stakeholders, the CPO team functioned as scouts to identify analytical resources that were required to support process stakeholders in identifying common and variable requirements for the platform. The CPO team researched various modeling mechanisms and industry practices that could be used: *“[CPO] team also looked at how the Truckload freight process could be defined using standard process steps [of SCOR] and activities as a way of developing the [OneBeta] process flowchart.”* [CPO member-2, Beta]

As the discussion on identifying common and variable requirements for their platform entered the critical stages of brainstorming, the CPO team performed the role of a knowledge integrator to enable seamless knowledge transfer and sharing. They channeled stakeholders to discover division-specific variable requirements by focusing on contextual details. In the end, as a result of these multiple roles that the CPO team adopted, agreement was reached among stakeholders on the common and variable requirements.

4.2.3. Changing Roles of Boundary Objects. The practice of *changing roles of boundary objects* highlights that the purpose served by boundary objects in PFD is evolving. At Beta, the CPO team had to constantly evolve boundary objects from simple process-modeling templates to more comprehensive contextualized knowledge capture tools to enable the visualization of common and division-specific variant requirements.

At Beta, although the initial templates created by the CPO team and the SCOR standards defined the scope of the modeling effort, process stakeholders were unable to understand each other's processes and how these would be supported in the common platform. During one of the workshops, the CPO team realized that this difficulty stemmed from the use of significantly different transportation planning terminology across the divisions to refer to similar planning tasks. For example, Division 1 and Division 3 had very different usages of the term MAX rate. In Division 1, load planners set up a dummy carrier called MAX in their transportation planning system for every route whose rate is the average rate for the route to ensure that the optimization system will always try to find the lowest rate carrier. When load optimization selects this carrier for a given load, the load planner is alerted and he or she can then manually identify the best alternative carrier. In contrast, in Division 3, MAX rate indicates the highest possible rate paid to carriers for a given load and was determined on a load-to-load basis. This example highlights how the lack of a common vocabulary among different divisions clouded the common thread underlying the seemingly diverse planning processes: *“What we are trying to figure out . . . is how closely [planning activity] fits in with the other divisions? That one is so not close so far.”* [Process stakeholder-1, Beta] As the process stakeholders continued to reconcile diversity in the terminology

used, they realized that it was virtually impossible to make use of identical terminology to refer to similar process concepts across multiple divisions. Instead, they decided that it would be more useful to agree on a mapping across various terms to develop a shared understanding of the terminology used. The terms in the vocabulary were organized by the transportation tasks identified in the SCOR process. *“Use of the vocabulary is one way to achieve the connection in our processes and avoid some of our confusion.”* [CPO facilitator, Beta] Thus, although the boundary object was initially developed to provide a glossary that would help different parties agree on the definitions of various terms used, it was later augmented with semantic knowledge that enabled the identification of differences and commonality across the various divisions. Thus, the role of the boundary object changed from acting narrowly as a glossary to later serving as a means to understand commonality and differences in processes across divisions.

4.2.4. Situated Balancing of Boundary Spanner Roles and Boundary Objects. The practice of *situated balancing of boundary spanner roles and boundary objects* highlights how boundary spanner roles (such as a scout and knowledge integrator) were balanced with the use of boundary objects (such as SCOR, modeling techniques, and vocabularies) that augmented the role of the boundary spanners.

At Beta, after the CPO team brought in the concerned process stakeholders for negotiating common and division-specific variable requirements, they acted as scouts looking for mechanisms to facilitate a shared understanding of the common transportation planning platform. For example, in the first workshop, it was clear that stakeholders were quite inexperienced in modeling (in terms of what should be included/excluded, level of details, format to use, etc.). In the absence of standardized process models, discussions were unproductive, leaving stakeholders looking for more guidance and concrete examples of planning tasks modeled using the template: *“If someone else... has already done this, but ... in [a] different kind of format, just give us your format. We’ll plug in what we got... rather than recreate it from scratch.”* [Process stakeholder-2, Beta] Thus, it was clear that the CPO team’s scout role needed to be complemented with boundary objects to provide concrete examples of how to codify process knowledge. In another example of a balancing of boundary objects and boundary spanner roles at Beta, the CPO team further enriched the semantic boundary objects by capturing contextual knowledge in the form of business rules. These enriched boundary objects enabled stakeholders to interpret others’ planning processes with relative ease, differentiate their business practices from those of others, and identify common and variant requirements. This suggests explicit knowledge represented in boundary objects complemented boundary spanner roles.

This example is similar to the relatively informal boundary spanning that occurred at Alpha between developers in various application engineering teams. Several product derivations created to address client-specific needs had, in fact, some common features. However, application engineering teams often tacitly held knowledge about these. Instead of a formalized process for enabling knowledge sharing across application engineering teams, Alpha relied on informal boundary spanning to achieve shared understanding and reuse across product derivations. Developers in various application engineering teams spanned boundaries in an ad hoc manner to enable knowledge sharing and reuse.

4.2.5. Boundary Spanning to Stabilize Boundary Objects. The practice of *boundary spanning to stabilize boundary objects* highlights how certain stakeholders took on boundary spanning roles with the goal of stabilizing some boundary objects. Although the intent

was to develop a balanced and stable platform, the means to that end was considered to be developing a boundary object that all stakeholders could reach consensus on.

At Beta, the CPO team developed and implemented several strategies in its effort to elicit platform requirements from the stakeholders of different business divisions. Its goal was to identify common requirements of the platform to support the common planning process and business exceptions in the form of variable requirements unique to each division. Each of the business divisions described their transportation planning processes in the first workshop through process models of different levels of granularity and using different modeling notations and formats. For example, one division created straw-man models using Microsoft PowerPoint that depicted the entire planning process in just one slide, whereas another created detailed step-by-step illustrations using flowcharts that spanned multiple pages. *“What we reviewed today with business divisions shows the wide variation in detail . . . we need to talk about . . . the template, . . . the standardized format that we’re going to use to [model your processes]. Put them all in the same format, so we all are talking about same [level]. . . . We’ll need to identify the [common] functions.”* [CPO facilitator, Beta] These differences caused the discussions to remain at a superficial level, thus making it impossible to identify common processes, systems, data flows, and the like across divisions. To facilitate discussions, it was clear that process models needed to be developed using a common modeling formalism. The CPO team developed a template for process documentation based on activity diagrams and the “swim lane” format.

In the next workshop, the CPO team realized that, despite the use of the activity diagram with swim lanes, there were wide variations in the granularity with which stakeholders prepared their process models. As the CPO team urged stakeholders to create detailed diagrams, stakeholders became frustrated and raised concerns about the appropriate level of detail at which process models had to be developed. To address this, the CPO team suggested that an industry standard SCOR model for supply chain management be used to identify key processes in transportation planning. This standardization of process modeling helped process stakeholders develop an initial set of models for the family of planning processes and the underlying systems used across multiple divisions.

At Alpha, the requirements specification and architecture for the platform served as good boundary objects that helped in spanning the boundary between the domain and application engineering teams. However, to understand the extent of commonality of requirements and the extent of impact of changes to the platform on various customer segments, boundary spanning and boundary objects were used in conjunction with one another. The intent of boundary spanning was to stabilize the boundary object (here requirements specifications and architecture of the platform).

4.2.6. Improved RE Process in Conventional PFD through Boundary Spanning. The adapted boundary spanning practices used in the conventional PFD cases (Beta and Alpha) address some of the RE challenges identified earlier.

—Improved requirements elicitation: The practice “Changing roles of boundary spanners” addresses the challenge in engaging the customer base by providing mechanisms to help the various stakeholders gain insights into commonality and variability in the product family. The following quote highlights this: *“So basically the first thing we got through the client is . . . all the base functionalities the client can use. Once the client gets a picture of the base functionalities, then he might [say] ‘Ok, I am interested in that functionality. . . . I need to incorporate that functionality which speeds up my work or I like that functionality—we didn’t know that there is such a functionality existing before. . . . Now we know that there is a functionality and so can make use of*

- it.” [Manager, Alpha] This helps bridge the knowledge boundary pertaining to the fault line between customers and development teams.
- Improved requirements analysis: The practice “Creating controlled boundaries” addresses requirements analysis issues in that it enables domain and application engineering teams to arrive at a good understanding of what is common and what is variable in the product family. It also helped the customer understand how the platform would be defined and how it could be customized in product variants. A business analyst at Alpha comments: *“Developers working for different clients suggest elevating certain functionalities to be made as part of the base product. They have their own justifications for suggesting so.... If someone knows that this is ... asked [for] repetitively, then we suggest [that] the base team, the product development team ... incorporate the change that everybody is asking [for] so that it will be formed as a part of the base and there is no need for developing it again and again.”* [Business analyst, Alpha] This helps bridge the functional boundary pertaining to the fault line between the domain and application engineering teams.
 - Improved requirements specification: The practice “Situating balancing of boundary spanner roles and boundary objects” mitigates challenges in requirements specification by enabling tools and techniques used for modeling to become effective through appropriate boundary spanning. For example, at Beta, the *“[CPO] team also looked at how the Truckload freight process could be defined using standard process steps [of SCOR] and activities as a way of developing the [OneBeta] process flowchart.”*
 - The practice “Boundary spanning to stabilize boundary objects” addresses the challenges in requirements specification in that boundary objects that were used to model requirements needed to be stabilized, and that was done through boundary spanning. At Beta, the agreement to use a particular notation to model processes helped: *“The other thing to mention [is the charts] ... in swim lane format where functions are on the left and, as the process moves across, you can easily see which function is responsible for doing it today. I think that has some value. One of the other things is to agree on some kind of standardized function description.”* These two practices help bridge the knowledge boundary pertaining to the fault line between stakeholders from different divisions.
 - Improved requirements verification/validation: The practice “Changing roles of boundary spanners” addresses challenges in verification/validation by ensuring that the right requirements are incorporated in the platform and that the application engineering teams are deriving the right products from the platform. *“Generally, what happens is, I will not know how many other clients are having the same problem. If we see the problem [and realize that] it is a ... critical problem, that it is a base product, I will communicate that to the base team, and the base team will take the responsibility of sending emails to all project managers, telling [them] that if any your clients uses a specific process ... we have a critical fix that they need to have. So, the project manager then takes the responsibility of sending those fixes,”* states a business analyst at Alpha. This helps bridge the functional boundary pertaining to the fault line between the domain and application engineering teams.
 - Improved requirements management: The practice “Changing roles of boundary objects” addresses challenges in requirements management by providing adaptable boundary objects that help in understanding the impact of changes in requirements. The workshop facilitator from Beta explains: *“Basically what I am capturing is that the basic process of booking and tendering is the same as EDI except now it should be done in different ways for different divisions depending upon the circumstances in [the platform].”* This helps bridge the knowledge boundary pertaining to the fault line between process stakeholders in different divisions.

4.3. Boundary Spanning in Agile PFD

4.3.1. Evolving Boundary Objects. The practice of *evolving boundary objects* highlights the dynamic nature of boundary objects during the RE process used in the development of product variants. Project scope, content, and the level of details of boundary objects change during the RE process. In addition, boundary objects such as the requirements specifications also evolved continually during development.

Agile development teams in both organizations relied on ongoing negotiations with their respective customer representatives (product manager in Gamma, product owner in Delta) to identify the unique requirements for their product variants. Boundary objects such as requirements specifications that facilitate interactions between customers and developers and among development team members were evolving over time. *"About 50–60% of the initial requirements changed,"* stated a manager at Gamma. The experience at Delta was similar. *"We didn't have a thorough understanding of the market segment. ... We had to change our application significantly. ... Our user stories, therefore, were evolving,"* states the product owner at Delta.

During the initial product development cycles, the team members were more focused on developing a good understanding of the critical features of the application and the customization needed when adapting the platform rather than on creating detailed specifications of requirements. These high-level requirements specifications served as boundary objects that facilitated boundary spanning between the platform and the product variants. However, these boundary objects were dynamic in that they were changing in both scope and the level of detail. Agile requirements analysis continued at each development cycle during which features that needed to be incorporated in the product variant were identified and implemented. *"We took a radically different approach to creating this product. We didn't know fully what we were going to implement,"* explains a developer at Delta. At the beginning of each cycle, the customer sat down with the development team to provide detailed information on a set of desired features.

Because RE activities were intertwined with design and development activities, the outcomes were a set of finely defined requirements, a preliminary design, and sometimes an implementation plan, all of which served as important boundary objects. *"We were working on extending a complex platform. We had to use a design that is consistent with the design of the platform. We had to do it within the sprints so that we did just enough design for the current cycle,"* elaborates another developer at Delta. The unstable nature of the environment also contributed to evolving boundary objects. The designs changed over time as a result of both changing requirements and rapidly evolving technology.

4.3.2. Punctuated Change of Boundary Objects. *Punctuated change of boundary objects* emphasizes the balance between the stability required in PFD and the flexibility required in agile development. While boundary objects in agile development are evolving during RE and throughout the development of the product variant, the platform was insulated from frequent changes to provide the stability required by PFD.

As discussed in the practice "evolving boundary objects," boundary objects such as the specifications of product variants underwent numerous changes in agile PFD. These changes sometimes suggested a need to change the platform. However, frequent changes to the platform were not desirable. Therefore, boundary objects such as the platform requirements specifications underwent punctuated changes in that changes were done intermittently in a controlled manner. A manager at Gamma explained this: *"The platforms were updated periodically as they had to be stable."* Only the changes that were evaluated to be stable and common across multiple projects were incorporated into the platform. *"The product changed quite a bit. Therefore, we felt it was*

unnecessary to keep documenting all the designs and specifications when we knew that it would change. The liaison from the platform, therefore, was very careful to document only changes that remained stable. Of course, you don't know whether something will remain stable for the entire product development cycle. He had to make that judgment and often he was quite correct," explains the project manager at Delta.

Also, punctuated changes included changes in the meanings of boundary objects in different development phases to provide sufficient stability and reduce the overhead in maintaining documentation. For example, requirements were initially defined broadly with the intent of setting high-level goals that are shared between the team and customer. Such goals were later translated into finer grained specifications of requirements that were used to guide the developers in the design and implementation of system functionalities. However, requirements documents were not formally updated immediately when changes were made. After the features were finally implemented, corresponding specifications were recorded and tracked. *"This after-the-fact documentation of implemented features is quite different from starting with requirements and implementing them,"* explains the project manager in highlighting the differences between conventional and agile PFD.

4.3.3. Balanced Formality of Boundary Objects. *Balanced formality of boundary objects* highlights the balance between formal boundary objects needed for PFD and informal boundary objects used in agile RE process. In the RE process used by Gamma and Delta, boundary objects such as product/process repositories, requirements documents, product and sprint backlogs, and development procedures were used. In addition, other boundary objects such as design models and feature models of the product platform were also used to identify and articulate those features that were identified for implementation in various development cycles. These boundary objects were defined at a level of formality just sufficient to satisfy organizational norms without being burdensome to create and maintain.

Gamma and Delta adopted certain formal boundary objects. Gamma used predefined formal requirements documents to enhance knowledge sharing. Delta developed a database *"that helped the project team track open issues and project status. The product backlog was specified in short user stories. We didn't create elaborate use cases that are common in platform development."* These boundary objects were created using different formats and at different levels of formality to suit the context. Also, product managers in Gamma and the product owner in Delta were encouraged to clarify requirements by sketching out acceptance tests that served as formal boundary objects.

Although Gamma and Delta had both created formal boundary objects, they were cognizant of the need for flexibility in addressing specific project needs. They improvised several RE practices to fit their project context and needs. For example, the first two iterations at Delta were devoted to finalizing critical requirements and developing a high-level architecture. The team created detailed use cases for a few important requirements but kept the specifications of other requirements simple. Similarly, the first sprint at Delta helped *"understand at a high level what the customer segment wanted in terms of critical features. We weren't creating the traditional deliverables like we do in typical agile projects. The initial sprint was all about identifying the features of the platform that had to be extended or customized, helping the product owner prioritize the critical features and understanding them,"* explains the scrum master.

Although the conventional notion of boundary objects suggests a weak structure in common use and a strong structure in individual use, our observations indicate that RE processes are structured strongly centrally at the platform level and weakened in variant application (i.e., such structures are adapted to lighter structures for use in agile PFD environments).

4.3.4. Decentralized Boundary Spanner Roles. The practice of “Decentralized boundary spanner roles” focuses on the roles of boundary spanners in agile PFD that were unique and different from typical stereotypes observed in other contexts. Boundary spanner roles are decentralized to include all team members in agile PFD.

At both Gamma and Delta, each team member was empowered to serve as a boundary spanner by initiating and maintaining the flow of information across boundaries. For example, developers usually communicated directly with customers without going through the project manager. This type of decentralized boundary spanning was essential. *“We had excellent access to the product owner. It was essential given the complex nature of the product. We could communicate with him almost any time we needed to,”* explains a developer at Delta.

The decentralized boundary spanner roles changed the nature of boundaries between teams during the RE process. The intense collaboration between customers and developers blurred the boundaries because customers were considered members of the project team. *“The customer representative was with us all the time, he was a member of the team, and he had a cubicle right here,”* says a developer at Gamma. They used several communication channels to enhance the boundary spanning process. Daily meetings were used to share project information and facilitate common understanding among team members and between the developers and the customer. The blurred boundaries also affected the roles of the boundary spanners. *“Since customers were becoming part of our team, a lot of documentation was not used, and I didn’t have to be involved in all the communications between the members and the customer, or other activities that help them understand each other better,”* explains a project manager at Gamma.

4.3.5. Situated Balancing of Boundary Spanner Roles and Boundary Objects. “Situated balancing of boundary spanner roles and boundary objects” emphasizes the importance of boundary spanner roles in agile PFD while acknowledging the role of boundary objects as well. Whereas boundary objects facilitated communication and knowledge sharing, RE in agile PFD relies more on boundary spanner roles for boundary spanning and control.

Both Gamma and Delta used self-organized teams. Project managers provided sufficient autonomy and empowered employees to perform job-related activities. In Gamma, constant communication was considered essential to maintaining agility even with pre-specified initial requirements. *“While the requirement document was used to help communicate and clarify user requirements, we are now communicating with the customer face to face, which is extremely important.”* Intense collaboration with the customer helped the project manager better understand the customer’s immediate and longer term goals. The balance between the use of boundary objects and boundary spanners appeared to shift more toward the latter when agile methods were introduced.

Agile RE also relied more on boundary spanner roles than on boundary objects to provide the controls needed in PFD. For example, project managers who served in boundary spanner roles focused on providing tracking mechanisms to avoid the loss of critical information and to facilitate lateral communication among team members across boundaries. At Gamma, all important decisions and issues were required to be channeled through a lead developer who was designated as the “gate keeper” to make sure that critical decisions about the requirements made by the boundary spanner roles were tracked.

4.3.6. Improved RE Process in Agile PFD through Boundary Spanning. The boundary spanning practices in agile RE just described address some of the RE challenges discussed earlier and result in improved RE process quality:

- Improved requirements elicitation: The practice of “decentralized boundary spanner roles” facilitated constant and direct communication between team members and customers. In addition, on-site customer representatives significantly improved the user–developer interaction. Similarly, the practice of “situated balancing of boundary spanner roles and boundary objects” improved the involvement of the customer because boundary spanning relied more on the interaction between boundary spanners than on boundary objects. *“We were true partners. We were willing to listen to each other, keep each other in the loop about changes that may be coming up, difficulties faced in implementing features and so on,”* explains the project manager about his relationship with the product owner at Delta. These two practices helped span the knowledge boundary pertaining to the fault line between the customers and the development team. The practice “evolving boundary objects” also facilitated the emergence of new requirements in an iterative manner throughout the development process. This practice helped span the functional boundary pertaining to the fault line between team members playing different roles in the team.
- Improved requirements analysis: The practice of “evolving boundary objects” helped to improve how requirements were frequently prioritized and iteratively analyzed in each development cycle. Crucial requirements were identified by the customers and re-evaluated at each development cycle. *“I actively worked with the team, made it clear to them what the customer... wanted the most. Sometimes, my understanding of that was incomplete and changing. But the development team was willing to work with me and accommodate all my priorities,”* states the product owner at Delta. This practice helps span the knowledge boundary pertaining to the fault line between customers and the development team. The practice of “punctuated change of boundary objects” mitigated the risk of losing the distinction between platform and variant requirements. The platform requirements were only updated periodically after careful evaluation, leading to a better management of variation points and thereby achieving both flexibility and stability in RE. This practice helps span the functional boundary pertaining to fault lines between team members playing different roles in the team.
- Improved requirements specification: This was achieved by the practice of “balanced formality of boundary objects.” The risk of presenting requirements in the form of design was mitigated by avoiding the development of detailed models up front. Instead, requirements were allowed to iteratively evolve over time and were identified in the cycle in which they were implemented. A developer at Gamma explained: *“Design is delayed until we have thorough understanding of [those] requirements, which is at the design meeting in the beginning of each iteration. But both the requirements and design are evolving during the whole development process.”* On the other hand, certain formal boundary objects, such as a database of issues and initial requirements documents, were used to manage variable requirements and provide traceability between variable requirements and common platform requirements. This practice helps span the functional boundary pertaining to the fault line between team members playing different roles in the team.
- Improved requirements validation: This was achieved by the practice of “evolving boundary objects.” Requirements validation was not treated as a specific activity conducted during periodic reviews. Instead, requirements validation involved the inspection of not only user stories, but also of other boundary objects such as designs and features created at each development cycle. The frequent feedback from the customer obtained during and after each development cycle provided effective validation and verification of requirements. *“The product owner [customer] prioritized the needs of the project at each sprint. This may involve changing what we had implemented and, in fact, the customer had considered as ‘done,’”* states the project manager at

Delta. This practice helps span the knowledge boundary pertaining to the fault line between customers and the development team. The complexity involved in the dependencies across variable and common requirements was reduced by the practice of “punctuated change of boundary objects.” The validation of the consistency between variable and common requirements was conducted many times during application development. The scope and complexity of requirements that needed to be validated was thus reduced to a manageable level. This practice helps span the functional boundary pertaining to the fault line between team members playing different roles in the team.

- Improved requirements management: This was achieved by the practices of “punctuated change of boundary objects” and “evolving boundary objects.” The volatility of platform and application requirements was managed through the two practices. Requirement changes were incorporated based on constant negotiation with the customer. The requested changes to the platform were evaluated and documented before being incorporated into the platform. *“We didn’t spend much effort documenting the new features, but we had a liaison to the platform team who was charged with doing just that. . . . We were all focused on delivering [the product], but the document that the liaison was creating helped us understand how responsive we had been to satisfying the client even though their needs were constantly changing,”* explains a project manager at Delta. These two practices help span the knowledge boundary pertaining to the fault line between customers and the development team.

Our findings suggest that boundaries are shaped and spanned differently in conventional and agile PFD. Our findings also expand on the roles of boundary objects and spanners and how these are adapted to address RE challenges in these two environments.

5. DISCUSSION

Our findings highlight how PFD organizations adapt boundary spanning practices to address RE challenges in two development approaches. Table IV presents a comparison of conventional and agile PFD in terms of the various characteristics of boundary spanning: (i) fluidity of boundaries, (ii) formality of boundary objects, (iii) formalization of boundary spanner roles, (iv) complementarity of boundary objects and boundary spanner roles, (v) nature of knowledge, and (vi) knowledge processes enabled by boundary spanning.

Fluidity of boundaries is different across conventional and agile PFD in that the boundaries in the former are more pronounced, with an intent to separate concerns, whereas those in the latter are more fluid. Boundary objects are more formal in conventional PFD, whereas agile PFD attempts to balance the level of formality. Boundary spanner roles are more formalized and multidimensional in conventional PFD but decentralized in agile PFD. Although boundary objects complemented the role of boundary spanners in conventional PFD, boundary objects were considered substitutable with boundary spanners in agile PFD. Whereas knowledge about specific details about requirements and design was considered critical in conventional PFD, the rates at which the knowledge changes and how often such changes should be captured in boundary objects were critical in agile PFD. All three knowledge processes (transfer, translation, and transformation, as discussed in Carlile [2002, 2004]) were important in conventional PFD in that not only did different parties have to use a shared lexicon, but they also had to deal with the varied interpretations and interests of those involved. Agile PFD required knowledge translation to develop a shared understanding of requirements and knowledge transformation in helping address conflicts between different stakeholders.

Table IV. Characteristics of Boundary Spanning across Conventional and Agile PFD

Characteristics of boundary spanning	Conventional PFD	Agile PFD
Fluidity of boundaries	Conscious efforts are made to recognize boundaries between different stakeholder groups in an effort to maintain a stable base platform.	Boundaries between different stakeholders are fluid in that customers appear as part of the development team and the team is cross-functional.
Formality of boundary objects	Detail-oriented boundary objects that are represented formally to make knowledge explicit. Goal of these boundary objects is to help clarify boundaries across stakeholders.	While agility prompts the teams to create informal boundary objects, a balance in formality is considered important due to the need to ensure a stable platform.
Formalization of boundary spanner roles	Boundary spanner roles are designated but people enacting those roles started playing additional roles so as to ensure that unique requirements (variable) are handled while maintaining platform stability.	Boundary spanner roles were decentralized in that each member can adaptively play various boundary spanner roles.
Complementarity of boundary objects and boundary spanner roles	Roles of boundary spanners are as critical as the boundary objects. Boundary objects are not seen as substitutes but are considered as complements to boundary spanners. Boundary spanners use various boundary objects to traverse boundaries smoothly.	Boundary spanner roles were considered more critical than boundary objects and were also observed to be substitutable wherein boundary spanners take on roles that would have otherwise fulfilled by boundary objects.
Nature of knowledge	Complexity and granularity: PFD required complex knowledge that entailed dependencies across knowledge elements, and granular knowledge with a focus on specific details that are necessary for development teams to understand any changes to be made.	Volatility and representation: The speed of knowledge is transferred and the formality of knowledge representation were critical. Knowledge artifacts had to keep up with the rapid changes in requirements.
Knowledge processes enabled by boundary spanning	Boundary spanning-enabled knowledge transfer helped stakeholders converge on common notations, knowledge translation helped stakeholders from different domains gain a shared understanding, and transformation was critical where the different interests of the actors posed challenges (for example, customers, domain engineering teams, application engineering teams).	Boundary spanning-enabled knowledge translation helped developers and customers gain a shared understanding about the requirements and the business value of the application. Knowledge transformation helps solve the conflicts between different stakeholders, such as conflicts between customer and the team.

Next, we discuss the implication of our findings. We organize these implications into three key themes that emerged from our findings—(i) boundary spanning-enabled RE, (ii) adapted boundary spanning, and (iii) nuanced view of technical debt—all shaped by boundary spanning in PFD.

5.1. Boundary Spanning-Enabled RE

We present boundary spanning practices as solutions to complex RE challenges in PFD. By establishing adapted boundary spanner roles, domain and application engineering teams were able to better understand different types of requirements. Balancing boundary objects and boundary spanner roles enabled the development team to achieve

an appropriate tradeoff between formally specifying requirements and informally discussing the details of the requirements.

Our findings indicate that RE research can benefit from drawing on the theory of boundary spanning. Prior research on requirements specification [Guinan and Faraj 1998] that associates visionary and guard processes with project performance provides the initial step in establishing a link between the literatures on boundary spanning and RE. Our study strengthens this connection by specifically identifying adapted boundary spanning practices in RE.

Boundary spanning practices for RE were developed through varying adaptations for agile and conventional PFD. It raises interesting connections to the notion of dependency between boundary spanners and boundary objects highlighted by Jonsson et al. [2009]. For example, boundary spanners in agile PFD were considered more critical than boundary objects, with the latter often being considered as substitutable with the former. However, agile PFD projects also incorporated boundary objects to provide a balanced approach that enabled their teams to quickly address changing customer needs by deriving specific product variants from the platform while also ensuring that the advantages of having a stable platform are not lost. Although changes and product derivations were done quickly, “just enough” knowledge documentation about these enabled platforms allowed application engineering teams to understand the impacts on the platform and how future changes would be handled in product variants.

The practices are considerably different from how boundary spanning practices were adapted to suit conventional PFD. Conventional PFD required RE practices to be considerably more formal. Increased restrictions were placed on how variable requirements affected the decisions pertaining to the platform. Although boundary spanner roles were indeed considered important, they were tempered with the need to “protect” the domain engineering team and, in turn, the platform from being overwhelmed by short-term needs. The longer term view taken by conventional PFD required the use of more elaborate boundary objects. However, the focal organizations did recognize the need to balance boundary spanner roles with boundary objects. For example, boundary spanners had to ensure that boundary objects were defined and used in a way that is consistent with the spirit of conventional PFD while recognizing the limits of boundary objects. They actively sought to complement boundary spanning roles to address any limitations in the ability of boundary objects to serve the needs of stakeholders.

The purpose of boundary spanning in terms of meeting the knowledge needs of the teams to help support RE differed between conventional and agile PFD. Whereas complexity driven by dependencies and specific details of requirements were important characteristics of knowledge shuttled across boundaries in conventional PFD, agile PFD teams were more concerned about rapid changes to knowledge and how boundary objects had to keep up with such changes. Whereas the knowledge processes supported by boundary spanning were similar across conventional and agile PFD, they were enacted differently, as highlighted in our findings.

5.2. Adapted Boundary Spanning

Prior research on boundary spanning has identified various roles that boundary spanners play in “providing communication linkages that organizational members maintain” [Pawlowski and Robey 2004]. Such roles have been empirically related to performance [Guinan and Faraj 1998]. Our findings suggest interesting extensions in these boundary spanner roles to support RE in complex software development efforts such as PFD projects. For example, the goal of developing a stable platform prompted the project teams to establish boundaries governed by certain principles. Specified communication channels and a process to request and include any requirement as a platform requirement were facilitated by the establishment of an approval process

and a boundary spanner role at the boundaries between customers, domain engineers, and application engineers. While this boundary spanner kept track of functionalities offered by the platform and ascertained whether additional requirements were consistent with these functionalities, he or she also ensured that the platform supported all the common functionalities needed by the variants. Although such responsibilities are seemingly similar to those attributed to the role of a broker [Pawlowski and Robey 2004], in our context, these were held by members of the domain engineering team. Such a role is complex and requires considerable interaction with other members of the domain engineering team and the examination of various artifacts used by this team (such as requirements specification, platform architecture, traceability matrices, feature models, etc.). Thus, the role of these boundary spanners is considerably more complex than simply filtering or facilitating.

Our findings also highlight interesting interactions between boundary objects and boundary spanners that enabled PFD teams to better manage their RE process. Some of our findings resemble Levina and Vaast's [2005] observation on the emergence of boundary spanners in practice in that some members of the domain engineering team ended up playing the role of boundary spanners to ensure the stability of the platform. However, our findings differ from Levina and Vaast's in two ways. First, the conditions that enabled some team members to become boundary spanners were different in our context. As specified in the "formalization of boundary spanner roles" in Table IV, in conventional PFD, team members who worked on specific features and boundary objects, such as specifications or traceability documentation, and those who were identified as responsible individuals served as boundary spanners. In agile PFD, all team members were considered boundary spanners because they directly interacted with other stakeholders, such as customer representatives. Second, a wide set of stakeholders were involved in developing and using boundary objects. Also, the boundary objects and boundary spanners played a complementary role in conventional PFD and a substitutive role in agile PFD as discussed in the "Complementarity of boundary objects and boundary spanner roles" characteristic in Table V. For example, members of the application engineering team had to evaluate whether some of the variable requirements could become platform requirements. In this evaluation, boundary objects that document and trace these requirements provide a starting point. In addition, spanning the boundary across various application engineering teams becomes necessary in this evaluation. On the other hand, when an application engineering team member needs details about a configuration in the platform that was incorporated to accommodate an anticipated variable requirement, she might immediately contact a domain engineer. The domain engineer then directs her to boundary objects that document the requirement. Thus, boundary objects and boundary spanners played a hand-in-glove role in enabling commonality and variability management in RE for PFD. In agile PFD, intensive communication between boundary spanners replaced the need to create formal and detailed boundary objects. Also, the notion of the balanced role of boundary spanners and boundary objects is interesting in the context of agile PFD. For example, Marrone et al. [2007] argue that teams are likely to be better off when boundary spanning responsibilities are spread across the entire team rather than being concentrated in a few individuals. This not only reduces individual role overload, but also develops a shared sense of accountability in the team. This is consistent with the spirit of "collective ownership" in agile methods and is particularly useful in agile PFD pertaining to the practice of decentralized boundary spanning. Marrone et al. [2007] further argue that any adverse outcomes of diffused responsibility can be countered by collective accountability. Our findings are complementary to this argument in that the balance between boundary spanning and boundary objects, the dispersed boundary spanning across agile PFD teams, and shared accountability all worked hand-in-glove.

5.3. Nuanced View of Technical Debt

IT projects incur technical debt when the developers “take shortcuts that fall short of best practices” [Allman 2012]. Allman argues that understanding, communicating, and managing technical debt is essential for the long-term success of a system. Whereas technical debt was initially conceptualized as producing immature code in order to deliver solutions to market needs [Cunningham 1992], a broader view includes the notion of tradeoffs between short-term decisions made during development and their potentially long-term costs [Klinger et al. 2011]. Thus, whereas both conventional and agile development approaches may accumulate technical debt, the issue is more pronounced and observed relatively more quickly in agile development due to its focus on time-to-market considerations. Specifically, in the context of PFD, given the significant investments made in the development of the product platform during domain engineering in conventional PFD—in contrast to the intense focus on time-to-market considerations in agile PFD—the problem of technical debt is more readily observed in agile PFD.

The concept of technical debt in agile development has been well recognized [Cao et al. 2010]. For example, inadequate refactoring of code results in the accumulation of technical debt to a point at which it is no longer feasible to proceed with development without major refactoring. The concept of technical debt appears to take on additional significance when agile methods are used in PFD. In the focal projects, the need to pay for technical debt was recognized at the outset. Unlike technical debt that accumulates as a result of practices used in the development process, the need to understand the functionalities of the platform before the development of a product variant can commence may be seen as a technical debt imposed on the development team by the development context. The focal projects, indeed, spent initial development cycles in boundary spanning activities to erase this technical debt. We argue that such a framing may help explain the need for boundary spanning activities when agile approaches are used in large, complex projects and maintenance activities that require up-front initial investment to develop an adequate understanding of the development context. Conventional PFD approaches, on the other hand, due to the efforts expended in developing a robust platform during domain engineering, seem to address issues related to technical debt more carefully. A recent study [Lim et al. 2012] suggests that managers are more willing to incur technical debt because of their focus on market considerations, whereas engineering teams are less willing to incur technical debt because of their focus on creating “perfect software.” This tension is indeed similar to the tension between agile and conventional PFD in terms of the approaches toward technical debt observed in our study. Furthermore, in agile PFD, the technical debt observed in our study may be characterized primarily as “strategic debt” [Tom et al. 2013], whereas the technical debt observed in our conventional PFD projects may be characterized as “tactical debt.” Furthermore, concerns about “incremental debt” resulting from inadequate refactoring were also observed to be more pronounced in agile PFD. Although our study highlights this contrast between traditional and agile approaches in terms of their focus on creating and managing technical debt, a more detailed study of these differences merits further attention, especially due to its impact on outcomes such as morale, productivity, quality, and risk [Tom et al. 2013]. Our study also suggest that whereas technical debt is often “invisible or impressionistic especially to non-technical stakeholders” [Klinger et al. 2011], the intense involvement of a critical nontechnical stakeholder (namely, the customer or customer representative) and the use of various boundary spanning practices alleviate issues related to managing this technical debt in this context.

In summary, our findings provide the following insights:

- Boundary spanning-enabled RE in PFD should take into consideration the various characteristics of boundary spanning practices (described in Table IV). Such characteristics constitute abstractions of specific practices and help us translate our findings for different contexts.
- RE practices in conventional and agile PFD provide insights on how boundary spanning practices are adapted. These provide interesting variations on boundary spanning practices from existing literature in this area.
- Balancing technical debt and credit in conventional and agile PFD through boundary spanning-enabled RE highlights implications for cross-pollination of practices across conventional and agile PFD.

6. CONCLUSION

6.1. Implications for Theory

Our findings highlight the differences between boundary spanning practices in conventional and agile PFD to address RE challenges. These differences in boundary spanning practices are distilled further by the six characteristics of boundary spanning that we identify in Table IV (fluidity of boundaries, formality of boundary objects, formalization of boundary spanner roles, complementarity of boundary objects and boundary spanners, nature of knowledge, and knowledge processes enabled by boundary spanning). Our findings explain how boundary spanning enables RE, how boundary-spanning practices are adapted in conventional and agile PFD, and how technical debt in PFD is managed through appropriate boundary spanning.

6.2. Implications for Practice

Our findings provide the following four implications for practice:

- (1) Project managers should attempt to balance the needs of domain and application engineering teams. Specifically, they need to help the two teams achieve a balance between the need for a stable and flexible platform and the need to quickly deliver product variants.
- (2) Project managers should be cognizant of the need to tailor boundary spanning practices to address RE challenges based on the development approach (agile vs. conventional). Specifically, they should be sensitive to the differences between conventional and agile PFD contexts that are described in Tables II and IV while developing tailored practices.
- (3) Project managers who designate boundary spanners and specify the types of boundary objects need to recognize the complementarity of boundary spanner roles and boundary objects.
- (4) Project team members should be cognizant of the nature of knowledge and knowledge processes that are supported by the boundary spanning practices explained in our findings. This helps those playing boundary spanner roles and those working with boundary objects to understand the goals they are trying to accomplish through boundary spanning.

6.3. Limitations and Future Research

Findings from our study should be generalized with caution. Although the framework we have developed is based on examining agile and conventional PFD projects and may be particularistic to these contexts, key components of the framework may be applicable to any PFD project. However, these need to be further validated through additional empirical studies. It should also be noted that we have studied the phenomenon of boundary spanning at the team level and that PFD might be a phenomenon that organizations might view as foundational and applicable at an organizational level.

Further studies are necessary to examine PFD and boundary spanning at multiple levels of analysis. The framework that we have developed provides the much needed theoretical foundation to develop large-scale empirical studies of the role of boundary spanning in RE. Further research in this area can focus on examining how specific aspects of agile methods could play different roles in PFD projects. Further work could examine the impact of boundary spanning practices in RE on those later in the development life cycle.

REFERENCES

- E. Allman. 2012. Managing technical debt. *Communications of the ACM* 55, 50–55.
- V. Alves, N. Niu, C. Alves, and G. Valença. 2010. Requirements engineering for software product lines: A systematic literature review. *Information and Software Technology* 52, 806–820.
- D. Ancona and D. Caldwell. 1992. Bridging the boundary: External activity and performance in organizational teams. *Administrative Science Quarterly* 37, 634–665.
- B. A. Bechky. 2003. Sharing meaning across occupational communities: The transformation of understanding on a production floor. *Organization Science* 14, 312–330.
- M. Bergman, K. Lyytinen, and G. Mark. 2007. Boundary objects in design: An ecological view of design artifacts. *Journal of the Association for Information Systems* 8, 546–568.
- J. M. Bhat, M. Gupta, and S. N. Murthy. 2006. Overcoming requirements engineering challenges: Lessons from offshore outsourcing. *IEEE Software* 23, 38–44.
- J. Bosch, G. Florijn, D. Greefhorst, J. Kuusela, H. Obbink, and K. Pohl. 2001. Variability issues in software product lines. In *Proceedings of the 4th International Workshop on Product Family Engineering*.
- L. Cao, B. Ramesh, and T. Abdel-Hamid. 2010. Modeling dynamics in agile software development. *ACM Transactions on Management Information Systems* 1, 1–26.
- P. Carlile. 2004. Transferring, translating, and transforming: An integrative framework for managing knowledge across boundaries. *Organization Science* 15, 555–568.
- P. R. Carlile. 2002. A pragmatic view of knowledge and boundaries: Boundary objects in new product development. *Organization Science* 13, 442–455.
- P. Clements and L. Northrop. 2002. *Software Product Lines: Practices and Patterns*. Addison-Wesley, Upper Saddle River, NJ.
- A. Cockburn and J. Highsmith. 2001. Agile software development: The people factor. *IEEE Computer* 34, 131–133.
- K. Conboy. 2009. Agility from first principles: Reconstructing the concept of agility in information systems development. *Information Systems Research* 20.
- K. Conboy and L. Morgan. 2011. Beyond the customer: Opening the agile systems development process. *Information and Software Technology* 53, 535–542.
- W. Cunningham. 1992. The WyCash portfolio management system. *SIGPLAN OOPS Messages* 4, 29–30.
- B. Curtis, H. Krasner, and N. Iscoe. 1988. A field study of the software design process for large systems. *Communications of the ACM* 31, 1268–1287.
- E. J. Davidson. 2002. Technology frames and framing: A socio-cognitive investigation of requirements determination. *MIS Quarterly* 26, 329–358.
- G. Davis. 1982. Strategies for information requirements determination. *IBM Systems Journal* 21, 4–31.
- T. Demarco and T. Lister. 2003. Risk management during requirements. *IEEE Software* 20, 99–101.
- M. Dodgson, D. M. Gann, and A. Salter. 2007. In case of fire, please use the elevator: Simulation technology and organization in fire engineering. *Organization Science* 18, 849–864.
- K. Eisenhardt. 1989. Building theories from case study research. *The Academy of Management Review* 14, 532–550.
- K. M. Eisenhardt and M. E. Graebner. 2007. Theory building from cases: Opportunities and challenges. *Academy of Management Journal* 50, 25–32.
- M. Eriksson, J. Börstler, and K. Borg. 2009. Managing requirements specifications for product lines—An approach and industry case study. *Journal of Systems and Software* 82, 435–447.
- A. Espinosa, J. Cummings, J. Wilson, and B. Pearce. 2003. Team boundary issues across multiple global firms. *Journal of Management Information Systems* 19, 157–190.
- L. Fleming and D. M. Waguespack. 2007. Brokerage, boundary spanning, and leadership in open innovation communities. *Organization Science* 18, 165–180.

- U. Gal, K. Lyytinen, and Y. Yoo. 2008. The dynamics of IT boundary objects, information infrastructures, and organisational identities: The introduction of 3D modelling technologies into the architecture, engineering, and construction industry. *European Journal of Information Systems* 17, 290–304.
- S. Gasson. 2005. The dynamics of sensemaking, knowledge and expertise in collaborative, boundary-spanning design. *Journal of Computer-Mediated Communication* 10.
- S. Gasson. 2007. Progress and breakdowns in early requirements definition for boundary-spanning information systems. In *International Conference on Information Systems*.
- Y. Ghanam, F. Maurer, and P. Abrahamsson. 2012. Making the leap to a software platform strategy: Issues and challenges. *Information and Software Technology* 54, 968–984.
- E. Guba. 1981. Criteria for assessing the trustworthiness of naturalistic inquiries. *Educational Communication and Technology Journal* 29, 75–91.
- P. C. Guinan, J. and S. Faraj. 1998. Enabling software development team performance during requirements definition: A behavioral versus technical approach. *Information Systems Research* 9, 101–125.
- O. Henfridsson and R. Lindgren. 2010. User involvement in developing mobile and temporarily interconnected systems. *Information Systems Journal* 20, 119–135.
- M. Jarke, P. Loucopoulos, K. Lyytinen, J. Mylopoulos, and W. Robinson. 2011. The brave new world of design requirements. *Information Systems* 36, 992–1008.
- K. Jonsson, J. Holmström, and K. Lyytinen. 2009. Turn to the material: Remote diagnostics systems and new forms of boundary-spanning. *Information and Organization* 19, 233–252.
- M. Keil, P. E. Cule, and K. Lyytinen. 1998. A framework for identifying software project risks. *Communications of the ACM* 41, 76–83.
- L. Kirsch and M. Haney. 2006. Requirements determination for common systems: Turning a global vision into a local reality. *Journal of Strategic Information Systems* 15, 79–104.
- T. Klinger, P. Tarr, P. Wagstrom, and C. Williams. 2011. An enterprise perspective on technical debt. In *Proceedings of the 2nd Workshop on Managing Technical Debt*. ACM, 35–38.
- B. Kovitz. 2003. Hidden skills that support phased and agile requirements engineering. *Requirements Engineering* 8, 135–141.
- R. E. Kraut and L. A. Streeter. 1995. Coordination in software development. *Communications of the ACM* 38, 69–81.
- D. Lau and J. Murnighan. 2005. Interactions within groups and subgroups: The effects of demographic faultlines. *Academy of Management Journal* 48, 645–659.
- B. Lawrence, K. E. Wieggers, and C. Ebert. 2001. The top risks of requirements engineering. *IEEE Software* 18, 62–63.
- A. Lee. 1989. A scientific methodology for MIS case studies. *MIS Quarterly* 13, 33–50.
- N. Levina and E. Vaast. 2005. The emergence of boundary spanning competence in practice: Implications for implementation and use of information systems. *MIS Quarterly* 29, 335–363.
- N. Levina and E. Vaast. 2008. Innovating or doing as told? Status differences and overlapping boundaries in offshore collaboration. *MIS Quarterly* 32, 307–332.
- E. Lim, N. Taksande, and C. Seaman. 2012. A balancing act: What software practitioners have to say about technical debt. *Software, IEEE* 29, 22–27.
- A. Malhotra and A. Majchrzak. 2012. How virtual teams use their virtual workspace to coordinate knowledge. *ACM Transactions on Management Information Systems* 3, 1–14.
- J. A. Marrone, P. E. Tesluk, and J. B. Carson. 2007. A multilevel investigation of antecedents and consequences of team member boundary-spanning behavior. *The Academy of Management Journal* 50, 1423–1439.
- J. Mason. 2002. *Qualitative Researching*. Sage Publications, London.
- B. Mcevily, V. Perrone, and A. Zaheer. 2003. Trust as an organizing principle. *Organization Science* 14, 91–103.
- M. Meyer. 2010. The rise of the knowledge broker. *Science Communication* 32, 118–127.
- M. B. Miles and A. M. Huberman. 1994. *Qualitative Data Analysis: An Expanded Sourcebook*. Sage, Thousand Oaks, CA.
- K. Mohan, B. Ramesh, and V. Sugumaran. 2010. Integrating software product line engineering and agile development. *IEEE Software* 27, 48–55.
- M. Moon, K. Yeom, and H. Seok Chae. 2005. An approach to developing domain requirements as a core asset based on commonality and variability analysis in a product line. *IEEE Transactions on Software Engineering* 31, 551–569.

- J. Nandhakumar, N. S. Panourgias, and H. Scarbrough. 2013. From knowing it to “getting it”: Envisioning practices in computer games development. *Information Systems Research*, 933–955.
- S. Nerur, R. Mahapatra, and G. Mangalaraj. 2005. Challenges of migrating to agile methodologies. *Communications of the ACM* 48, 72–78.
- W. J. Orlikowski. 1993. CASE tools as organizational change: Investigating incremental and radical changes in systems development. *MIS Quarterly* 17, 309–340.
- P. Padmanabhan and R. Lutz. 2005. Tool-supported verification of product line requirements. *Automated Software Engineering* 12, 447–465.
- F. Paetsch and F. Maurer. 2003. Requirements engineering and agile software development. In *12th IEEE International Workshops on Enabling Technologies: Infrastructure for Collaborative Enterprises* 308–313.
- S. D. Pawlowski and D. Robey. 2004. Bridging user organizations: Knowledge brokering and the work of information technology professionals. *MIS Quarterly* 28, 645–672.
- J. T. Polzer, C. B. Crisp, S. L. Jarvenpaa, and J. W. Kim. 2006. Extending the faultline model to geographically dispersed teams: How colocated subgroups can impair group functioning. *Academy of Management Journal* 49, 679–692.
- B. Ramesh, L. Cao, and R. Baskerville. 2010. Agile requirements engineering practices and challenges: an empirical study. *Information Systems Journal* 20, 449–480.
- E. Rubin and H. Rubin. 2012. Supporting agile software development through active documentation. *Requirements Engineering* 16, 117–132.
- S. A. Slaughter, L. J. Kirsch, L. Ma, S. R. Boss, E. I. Diamant, and M. H. Haney. 2007. The formation and evolution of faultlines in large-scale, multi-party information systems development. In *Proceedings of the International Conference on Information Systems*.
- A. Strauss and J. Corbin. 1990. *Basics of Qualitative Research: Grounded Theory Procedures and Techniques*. Sage Publications, London.
- E. Tom, A. Aurum, and R. Vidgen. 2013. An exploration of technical debt. *Journal of Systems and Software* 86, 1498–1516.
- D. M. Weiss and C. T. R. Lai. 1999. *Software Product-Line Engineering: A Family-based Software Development Process*. Addison-Wesley, Reading, MA.
- A. Yan and M.-R. Louis. 1999. The migration of organizational functions to the work unit level: Buffering, spanning, and bringing up boundaries. *Human Relations* 52, 25–47.
- R. K. Yin. 2003. *Case Study Research: Design and Methods*. Sage Publications, London.

Received April 2013; revised March 2014; accepted April 2014